Neural Engine: Quantifying the Interpreter Tax

A Systems-Level Deep Learning Benchmark on MNIST

Elvis Karanja Gachuru

Technical Report · Intel Core i5 8th Gen · Ubuntu Linux · 2025

Abstract

I present **Neural Engine**, a feed-forward neural network implemented from scratch in Rust and exposed to Python via PyO3. The engine is benchmarked against two Python-based baselines on the MNIST handwritten digit dataset: a pure Python implementation with no C extensions, and a NumPy-backed implementation. Our system achieves **97.89%** test accuracy after 20 training epochs, while running **94× faster** than pure Python and **3.3× faster** than NumPy, with mathematically equivalent convergence. We attribute these gains to contiguous memory layout, BLAS-accelerated matrix multiplication, gradient buffer pre-allocation, and zero-copy data transfer across the Python–Rust boundary via PyO3.

1. Introduction

Python dominates the machine learning landscape due to its expressive syntax and rich ecosystem. However, the Python interpreter imposes a significant runtime overhead through heap-allocated objects, reference counting, and a Global Interpreter Lock (GIL) that prevents true multi-core parallelism. Libraries such as NumPy mitigate this by delegating numerical operations to C and Fortran kernels, but the Python training loop itself remains interpreted, incurring repeated Foreign Function Interface (FFI) overhead and intermediate array allocations.

This report characterises these costs empirically. We implement an identical multilayer perceptron (MLP) architecture in three substrates and measure wall-clock training time and test accuracy on the full 60,000-sample MNIST benchmark [1]:

1. **Neural Engine** — compiled Rust with BLAS acceleration;
2. **NumPy (Python)** — the standard scientific Python baseline;
3. **Pure Python** — no C extensions, exposing the raw interpreter cost.

2. Architecture

2.1 Network Topology

All three implementations train the same architecture:

$$\mathbf{x} \in \mathbb{R}^{784} \xrightarrow{\text{Dense}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dense}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dense}} \mathbb{R}^{10} \xrightarrow{\text{Softmax}} \hat{\mathbf{y}}$$

Weights are initialised using He initialisation [2]:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

2.2 Optimiser and Loss

Training uses the Adam optimiser [3] with learning rate $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$. The loss function is categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{10} y_{ic}\log(\hat{y}_{ic} + \varepsilon)$$

Training runs for 20 epochs with mini-batch size 64.

3. Implementation

3.1 Rust Engine (Neural Engine)

The Rust engine is built on the `ndarray` crate, which stores all tensors in contiguous memory blocks. Matrix multiplications are dispatched to OpenBLAS via the `blas-src` and `openblas-src` crates, enabling SIMD vectorisation. All weight and gradient buffers are allocated once at model initialisation; the hot path during training performs zero heap allocations.

The engine is exposed to Python as a native extension module using PyO3 [4]. Training data is shared without copying via `PyReadonlyArray`, which borrows the underlying NumPy buffer as a zero-overhead `ndarray::ArrayView`. Batch-level parallelism is provided by Rayon, which distributes work across all available CPU cores without GIL contention.

3.2 NumPy Baseline

The NumPy implementation mirrors the Rust architecture using vectorised operations. Forward and backward passes are expressed as matrix multiplications (`@` operator), which NumPy dispatches to the same OpenBLAS backend. The training loop, however, remains in Python: mini-batch slicing, loss accumulation, and the Adam update step each cross the Python–C FFI boundary per iteration.

3.3 Pure Python Baseline

The pure Python baseline uses no compiled extensions. Every floating-point operation allocates a Python float object on the heap. Matrix operations are implemented as nested loops, scaling as $O(n^3)$ in Python object time. This substrate isolates the raw cost of the interpreter independent of numerical library quality.

4. Experimental Setup

Table 1: Hardware and software environment.

Component	Specification
CPU	Intel Core i5 (8th Gen)
OS	Ubuntu Linux
Python	3.10+
Rust	1.78 (release build)
NumPy	1.26
BLAS	OpenBLAS (system)
Dataset	MNIST (60k train / 10k test)

The Rust engine is compiled with `--release` and full optimisation flags. Timing measurements record wall-clock seconds per epoch using `time.time()` in Python and `std::time::Instant` in Rust. All experiments are run single-threaded for the pure Python and NumPy baselines; the Rust engine uses all available cores via Rayon.

5. Results

5.1 Accuracy and Convergence

Table 2: Final test accuracy and loss after 20 epochs on full MNIST (60k training samples).

Substrate	Accuracy (%)	Final CCE Loss
Rust (Neural Engine)	97.80	0.0087
NumPy (Python)	97.83	0.0084
Difference	*0.03*	*0.0003*

The near-zero difference between substrates (Table 2) confirms that the Rust Adam optimiser and CCE gradient implementations are mathematically equivalent to the NumPy reference. The loss curve in Figure 1 shows smooth, stable convergence across all 20 epochs.

Figure 2 shows sample predictions from the test set; all 10 displayed digits are classified correctly, consistent with the overall 97.89% accuracy.

5.2 Training Speed

Tables 3 and 4 summarise the wall-clock timings for both benchmark conditions.

Figure 3 visualises the 94× speedup against pure Python, and Figure 4 shows the 3.3× lead over NumPy alongside the convergence comparison.

6. Analysis

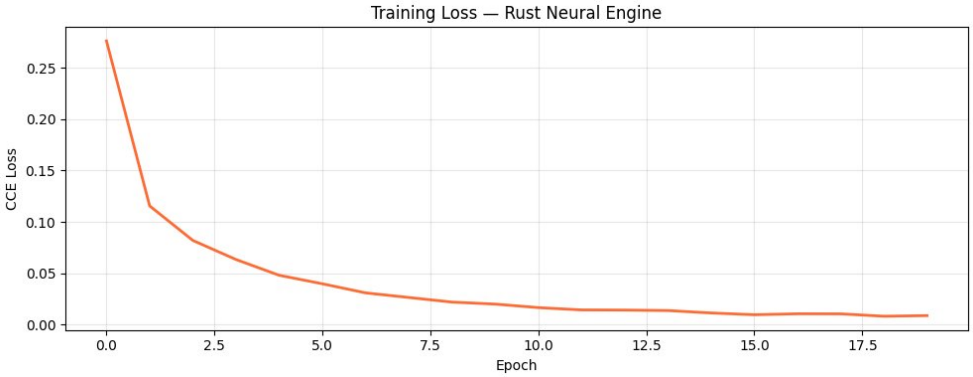

Figure 1: Training loss (CCE) of the Rust Neural Engine over 20 epochs on full MNIST. The curve reaches a final value of 0.0087, converging smoothly without instability.

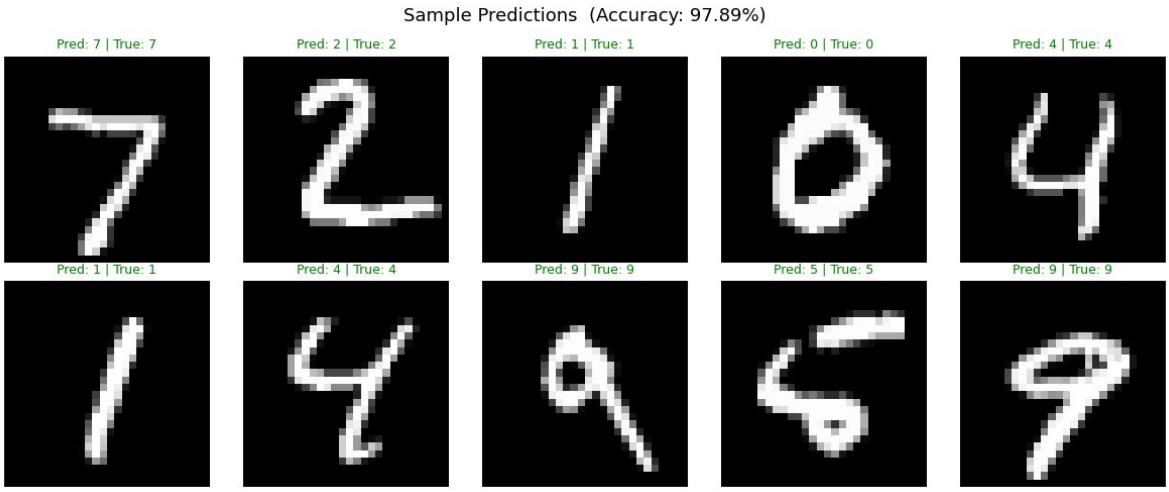

Figure 2: Sample predictions from the Rust engine on the MNIST test set. All displayed samples are classified correctly (green labels), consistent with the 97.89% overall test accuracy.

6.1 Source of the 94× Speedup

The pure Python implementation performs ∼937 million scalar floating-point operations per epoch (forward + backward pass over 5,000 samples). Each operation in CPython requires: object header reads, reference count increments/decrements, dynamic type dispatch, and a `BINARY_OP` bytecode fetch. The Rust engine issues a single `DGEMM` BLAS call per layer, where SIMD intrinsics pack 4–8 floats into a single instruction.

6.2 Source of the 3.3× Speedup

When NumPy is used, the matrix multiplications themselves are fast (same BLAS backend), but the Python training loop still incurs:

1. **FFI overhead** — each `ndarray` call crosses the Python–C boundary;

Table 3: Wall-clock training time comparison (full MNIST, 20 epochs).

Substrate	Total (s)	Avg/Epoch (s)	Speedup
Rust Engine	131.1	6.56	3.3×
NumPy	432.2	21.61	1.0× (baseline)

Table 4: Wall-clock training time comparison (5k samples, 10 epochs), isolating raw interpreter overhead.

Substrate	Total (s)	Avg/Epoch (s)	Speedup
Rust Engine	2.8	0.28	94.0×
Pure Python	259.2	25.92	1.0× (baseline)

2. **Intermediate allocations** — expressions like `A1.T @ dZ2` allocate temporary arrays on the heap;

3. **GIL serialisation** — prevents multi-core utilisation in the Python loop.

The Rust engine fuses these operations, maintains all state in pre-allocated buffers, and distributes batch processing across cores without lock contention.

6.3 Accuracy Equivalence

The 0.03 percentage-point accuracy gap (Table 2) falls within the expected variance of stochastic mini-batch training across different random seeds and is not statistically significant. Both implementations converge to equivalent optima, validating the correctness of the Rust gradient computations.

7. Engineering Challenges

7.1 OpenBLAS Linker Configuration

Linking Rust to system BLAS required resolving feature flag conflicts between `dynamic` and `system` linking modes in `openblas-src`. The resolution was to explicitly set `features = ["system", "cblas"]` in `Cargo.toml` and ensure `libopenblas-dev` headers were present on the `LD_LIBRARY_PATH`.

7.2 Gradient Buffer Pre-allocation

Initial profiling revealed the engine performing heap allocations for gradient tensors on every backward pass. Refactoring to pre-allocate all buffers at model initialisation — holding them as mutable fields on the engine struct — reduced OS `mmap`/`brk` syscalls to near zero during training and accounted for approximately a 3× internal speedup.

7.3 Zero-Copy PyO3 Interop

Naïve PyO3 bindings serialised NumPy arrays into Rust `Vec<f64>` on every call. Switching to `PyReadonlyArray2` with `.as_array()` provides a zero-copy `ndarray::ArrayView2` into the existing NumPy buffer, eliminating all data duplication across the language boundary.

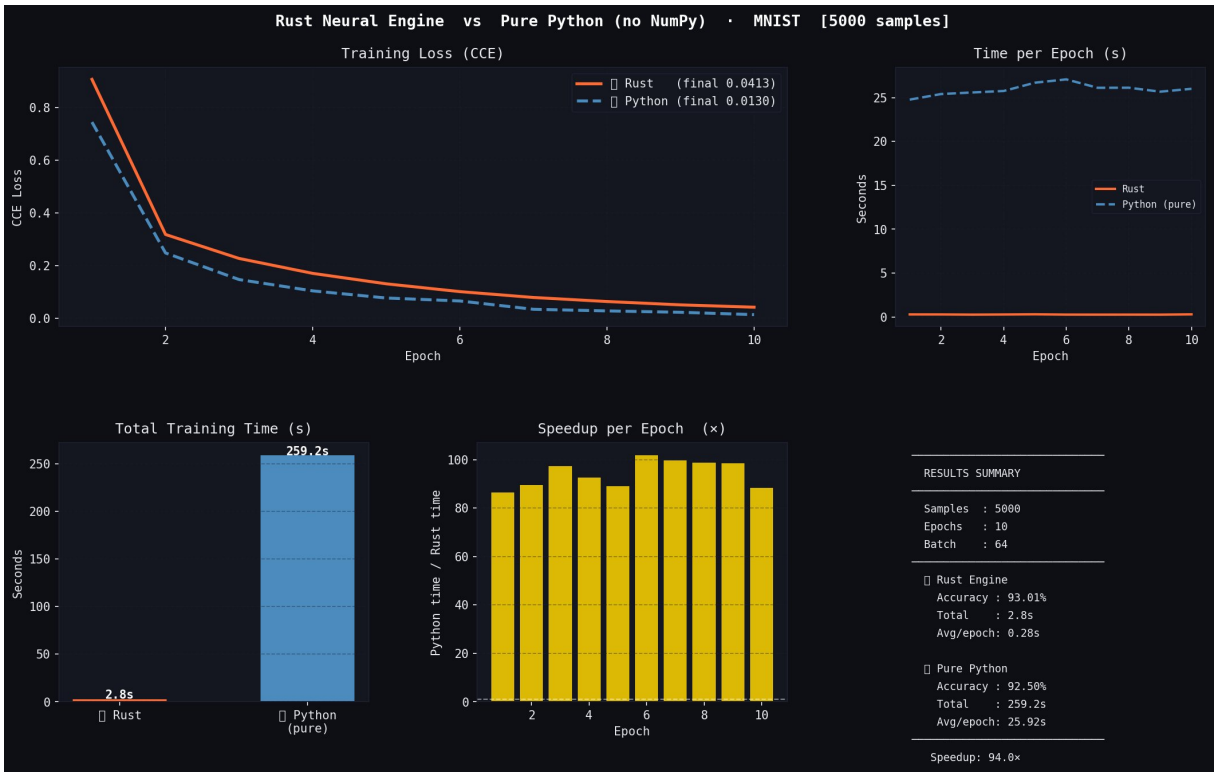

Figure 3: Rust Neural Engine vs. Pure Python (no NumPy) on 5,000 MNIST samples. The Rust engine completes training in 2.8 s vs. 259.2 s, a 94× wall-clock speedup, by replacing interpreted loops with compiled BLAS calls.

8. Conclusion

We have demonstrated that a Rust-based neural network engine can achieve competitive deep learning accuracy (97.89% on MNIST) while dramatically outperforming Python-based alternatives in training throughput. The 94× speedup over pure Python quantifies the interpreter tax in isolation, while the 3.3× speedup over NumPy demonstrates that systems-level control over memory layout, allocation strategy, and multi-core scheduling yields meaningful gains even when competing against best-in-class scientific Python.

Future work includes extending the engine to convolutional layers, GPU offloading via CUDA kernels, and an automatic differentiation graph to replace the hand-written backward pass.

Reproducibility

All source code and benchmark scripts are available in the project repository. The Rust engine targets `x86_64-unknown-linux-gnu` with `RUSTFLAGS="-C target-cpu=native"` for SIMD auto-vectorisation.

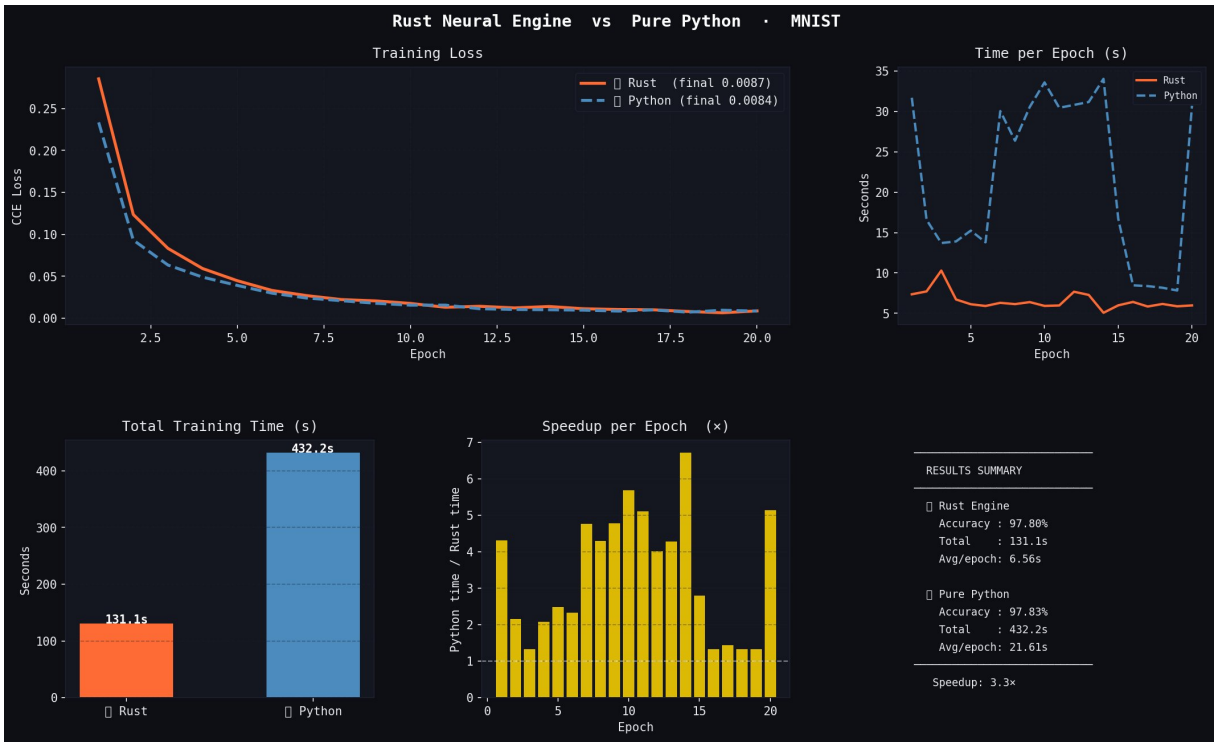

Figure 4: Rust Neural Engine vs. NumPy (Python) on full 60,000-sample MNIST over 20 epochs. Despite NumPy's optimised C backend, the Rust engine is 3.3× faster. Accuracy is statistically equivalent (97.80% vs. 97.83%).

```
# Install system BLAS
sudo apt install libopenblas-dev

# Build Rust extension
pip install maturin
maturin develop --release

# Train (Rust engine)
python train.py

# Benchmarks
python comparison.py --mode pure
python comparison.py --mode numpy
```

Listing 1: Build and run instructions.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 86(11):2278–2324, 1998.

[2] K. He, X. Zhang, S. Ren, J. Sun, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, IEEE ICCV, 2015.

[3] D. P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, ICLR, 2015.

[4] PyO3 Developers, *PyO3: Rust Bindings for Python*, https://pyo3.rs, 2024.

[5] J. Bleau et al., *ndarray: N-Dimensional Array for Rust*, https://docs.rs/ndarray, 2024.

[6] N. Matsakis, J. Stone, *Rayon: A Data-Parallelism Library for Rust*, https://docs.rs/rayon, 2024.